

Reproducing DOMINANT: Deep Anomaly Detection on Attributed Networks

A Reproduction Study and Debugging Analysis

Ali Bouzena

September 2026

Abstract

This report documents a full reproduction of DOMINANT [1], a graph autoencoder for anomaly detection on attributed networks, undertaken as an independent research exercise to build a deep, hands-on understanding of graph anomaly detection methods. The model was implemented independently in two ways — a from-scratch NumPy version with hand-derived, numerically verified backpropagation, and a PyTorch / PyTorch Geometric version evaluated against both a hand-built and a standard-library (PyGOD) implementation on a real benchmark dataset (`inj_cora`, from the BOND benchmark [3]). The final reproduction reaches an ROC-AUC of 0.783–0.786 across both independent implementations, consistent with the original paper’s reported range of 0.75–0.82. Two substantive bugs were identified and corrected during the process: an architectural class-imbalance failure in the structure-reconstruction decoder, and a silent label-encoding error in the evaluation pipeline. Beyond matching the paper’s headline numbers, the debugging process surfaced a genuine, dataset specific finding: the structure decoder does not detect the injected structural (clique) anomalies directly, but instead indirectly flags attribute-based anomalies through embedding corruption — a nuance the original paper’s clean two-decoder narrative does not surface.

1 Introduction

Graph anomaly detection (GAD) is the task of identifying nodes, edges, or subgraphs whose patterns deviate from the rest of a network, with applications ranging from fraud and intrusion detection to social-spam and citation-network analysis. DOMINANT [1] is one of the foundational deep learning approaches to this problem, combining a graph convolutional encoder with two reconstruction-based decoders. Its combination of relative simplicity and strong reported performance makes it a natural starting point for building a rigorous, hands-on understanding of the field, rather than relying on the paper’s description alone.

This report has two goals. First, to reproduce DOMINANT’s reported results as faithfully as possible, using two independent implementations for cross-validation. Second, and more substantively, to document the debugging process itself: several non-trivial issues were only discoverable by implementing the model directly, and the reasoning used to diagnose them is recorded here in full, since it reflects the actual research process rather than a polished after-the-fact summary.

2 Method Overview

DOMINANT is a graph convolutional autoencoder. An attributed network is defined by an adjacency matrix A and a node attribute matrix X . A stack of three Graph Convolutional Network

(GCN) layers [4] encodes every node into a low-dimensional embedding Z , blending each node’s own attributes with those of its neighborhood at increasing hop distances. Two separate decoders then attempt to reconstruct the original data from Z alone: a *structure decoder* that predicts the adjacency matrix as $\hat{A} = \sigma(ZZ^\top)$, and an *attribute decoder* — one further GCN layer — that predicts the original attribute matrix $\hat{X}$. The model is trained purely to minimize reconstruction error, with no anomaly labels involved at any point; anomalies are identified only afterward, as the nodes whose structure and/or attributes the trained model reconstructs worst. Figure 1 summarizes the architecture.

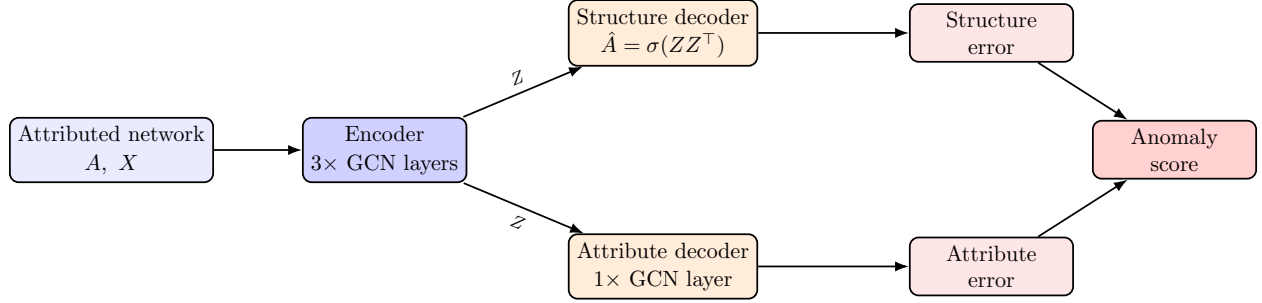


Figure 1: DOMINANT architecture: a shared GCN encoder produces node embeddings Z , which two independent decoders use to reconstruct structure and attributes; the combined, weighted reconstruction error becomes the per-node anomaly score.

The training objective combines both reconstruction errors with a weighting parameter α :

$$\mathcal{L} = (1 - \alpha) \|A - \hat{A}\|_F^2 + \alpha \|X - \hat{X}\|_F^2, \quad (1)$$

and the per-node anomaly score used for ranking is

$$\text{score}(v_i) = (1 - \alpha) \|a_i - \hat{a}_i\|_2 + \alpha \|x_i - \hat{x}_i\|_2. \quad (2)$$

3 Experimental Setup

Two independent reproduction pipelines were built:

1. **From-scratch NumPy implementation**, on a synthetic attributed network generated to mimic the paper’s community-structured social-network datasets, with structural (clique) and attribute-swap anomalies injected following the paper’s own Section 4.1 methodology. All backpropagation was hand-derived and verified against numerical finite-difference gradients before training (maximum relative error 1.3×10^{-8}), rather than relying on an autodiff library.
2. **PyTorch / PyTorch Geometric implementation**, evaluated on a real benchmark dataset, `inj_cora` — the Cora citation network with anomalies injected using the same methodology, packaged for standardized benchmarking by BOND [3]. This was cross-checked against an independent, standard-library implementation of DOMINANT from PyGOD [6].

4 Debugging Process and Findings

The synthetic NumPy run produced a clean result on the first attempt (ROC-AUC 0.775), because the toy dataset was simple enough not to expose any failure modes. The real benchmark dataset told a more instructive story, requiring two rounds of debugging before the results could be trusted.

Table 1: `inj_cora` dataset statistics.

Nodes	2,708
Edges	11,060
Attribute dimensions	1,433
True edge density	0.151%
Injected anomalies (total)	138 (5.1%)
– structural (clique)	68
– attribute (contextual)	68
– combined	2

4.1 Bug 1: an architectural floor on the structure decoder

The first full training run on `inj_cora` produced a training loss that was essentially frozen from epoch 1 onward (Figure 2, “before fix”), and an α sweep in which structure-only training ($\alpha = 0$) scored at exactly chance level while attribute-only training ($\alpha = 1$) was clearly best — the opposite of the mid-range sweet spot ($\alpha \in [0.4, 0.8]$) reported in the original paper. A diagnostic on the mean predicted edge probability, $\mathbb{E}[\hat{A}] = 0.500$, pointed directly at the cause: the encoder’s final layer applied a ReLU activation, which forces every embedding value to be non-negative. Since $\hat{A} = \sigma(ZZ^\top)$ and the dot product of two non-negative vectors is itself always non-negative, the model was *architecturally incapable* of predicting a connection probability below 0.5 for any pair of nodes — fatal on a graph that is 99.85% non-edges.

Removing the final-layer activation (matching the original Graph Autoencoder design of [5], which omits an activation on the embedding layer specifically for this reason) did not, by itself, resolve the issue: $\mathbb{E}[\hat{A}]$ remained at 0.502. The deeper cause was that plain mean-squared-error loss on a target that is 99.85% zeros gives the optimizer very little incentive to move away from a near-constant prediction, regardless of what values are technically reachable. The effective fix was to replace the structure loss with weighted binary cross-entropy, with the positive-class weight set to the ratio of non-edges to edges ($\approx 662:1$ on this graph) — the same remedy used in the original GAE implementation for exactly this class-imbalance problem. Figure 2 shows the effect: the mean predicted edge probability drops from 0.502 to 0.015, far closer to the true 0.0015, and Figure 3 shows the loss genuinely decreasing across training rather than being frozen.

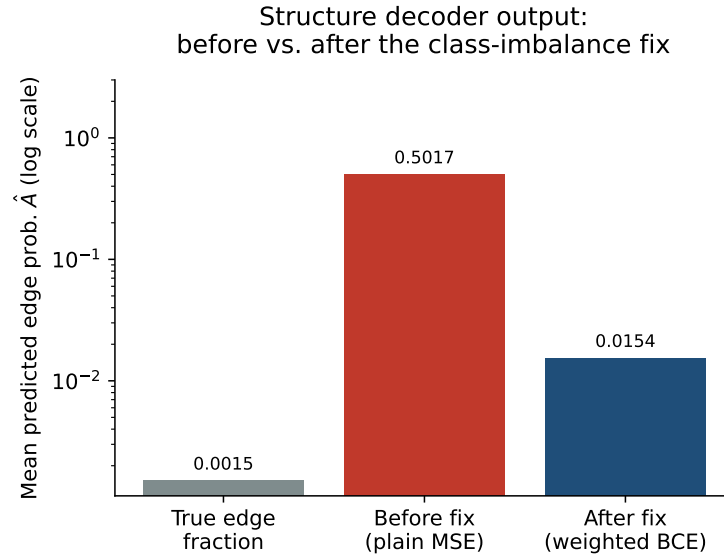


Figure 2: Mean predicted edge probability before and after switching to a class-weighted structure loss (log scale). Before the fix, the model collapses to predicting ≈ 0.5 for every node pair; after the fix, its predictions sit close to the true edge density.

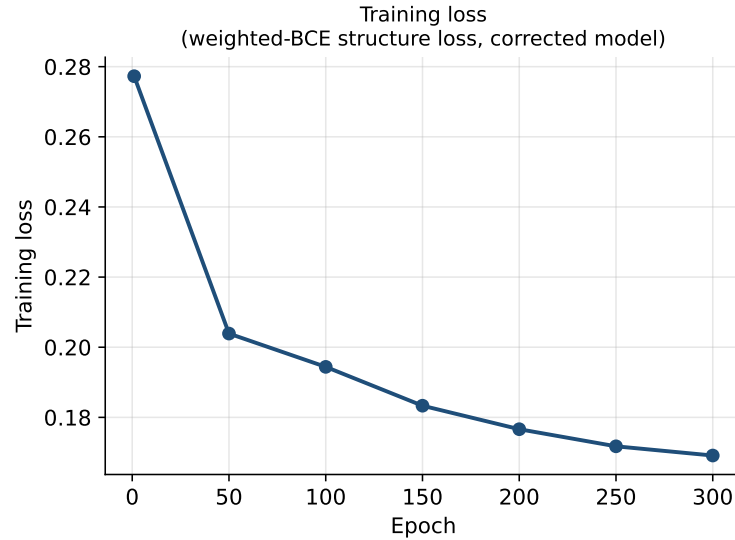


Figure 3: Training loss for the corrected model. Unlike the pre-fix run, which was frozen at ≈ 0.100 for the full 300 epochs, the loss decreases meaningfully throughout training.

4.2 Bug 2: a silent multi-class label error in evaluation

After fixing the structure decoder, overall ROC-AUC at $\alpha = 0.6$ *fell* to 0.62 and the α sweep grew worse rather than better — an unexpected result given the decoder was now behaving correctly. The cause was upstream of the model entirely. Inspecting the label array directly (`np.unique(data.y, return_counts=True)`) revealed four distinct values rather than the expected binary $\{0, 1\}$: label 0

for normal nodes (2,570), label 1 for structural anomalies (68), label 2 for attribute anomalies (68), and label 3 for combined anomalies (2) — the standard BOND injection encoding. The evaluation code, however, computed `binary_labels = (labels == 1)`, which counts only the 68 structural-type anomalies as true positives and silently treats the other 70 real anomalies as normal. Notably, the dataset-loading summary statistic “Anomalies: 210 (7.8%)” printed at the start of the notebook was itself an artifact of the same issue: `data.y.sum()` adds the label *codes* ($0 \times 2570 + 1 \times 68 + 2 \times 68 + 3 \times 2 = 210$) rather than counting anomalous nodes; the true count is $68 + 68 + 2 = 138$ (5.1%).

Correcting the evaluation to `binary_labels = (labels != 0)` — counting any injected anomaly type as positive — and re-evaluating the already-fixed model gives the results reported in Section 5. Figure 4 overlays the α sweep before and after both fixes, and is the clearest single illustration of the overall debugging process: the pre-fix curve is essentially flat and near-saturated (both bugs mask each other’s effects), while the corrected curve shows a real, informative relationship between α and performance, with structure-only training now scoring well above chance.

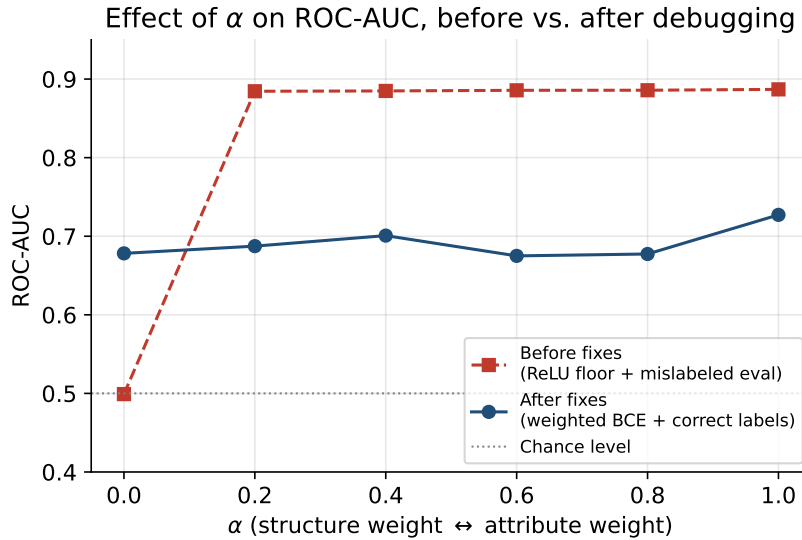


Figure 4: ROC-AUC as a function of α , before and after debugging. Before the fixes, structure-only training ($\alpha = 0$) scores at chance, an artifact of both the class-imbalance bug and the label-encoding bug combined. After both fixes, structure-only training is clearly informative ($\text{AUC} \approx 0.68$), and the gap to attribute-only training narrows substantially.

4.3 A finding that survives both fixes

One result did not change with either fix and is worth reporting on its own merits. Comparing structure-reconstruction error between true-normal nodes and true *structural-only* anomalies (label 0 versus label 1 specifically, excluding the other anomaly types entirely) shows no meaningful separation: mean structure error is 27.07 for normal nodes and 27.06 for structural anomalies, both before and after the class-imbalance fix. The structure decoder, even when correctly trained, does not learn to flag the artificially injected cliques on Cora specifically.

Its contribution to overall AUC instead comes almost entirely from indirectly catching the *attribute-type* anomalies: because the shared encoder blends attributes and topology together before producing each node’s embedding, a node with corrupted (swapped) attributes receives a distorted embedding purely from those bad attributes, which in turn also degrades the structure decoder’s

prediction for that node — despite its real connections never having changed. A plausible explanation for why the clique injection itself goes undetected is that real citation networks already contain naturally dense topical clusters, so an artificial clique may not be statistically distinguishable from a legitimate one, in a way it likely would be on a social-network graph such as BlogCatalog or Flickr, the datasets used in the original paper.

5 Results

Table 2 summarizes the final, corrected results from both independent implementations, alongside the original paper’s reported range on its own three datasets.

Table 2: Final reproduction results on `inj_cora`, versus the original paper’s reported range on BlogCatalog / Flickr / ACM.

	ROC-AUC	Precision@100	Recall@100
Hand-built (PyTorch + PyG)	0.7826	0.280	0.203
PyGOD (library)	0.7859	0.250	0.181
DOMINANT, original paper	0.75–0.82	–	–

The two independent implementations agree closely (0.7826 vs. 0.7859), which is stronger evidence of a correct reproduction than either result alone: a leftover implementation-specific bug would be expected to produce a larger discrepancy between two independently written models. Figure 5 compares Precision@K across both implementations directly.

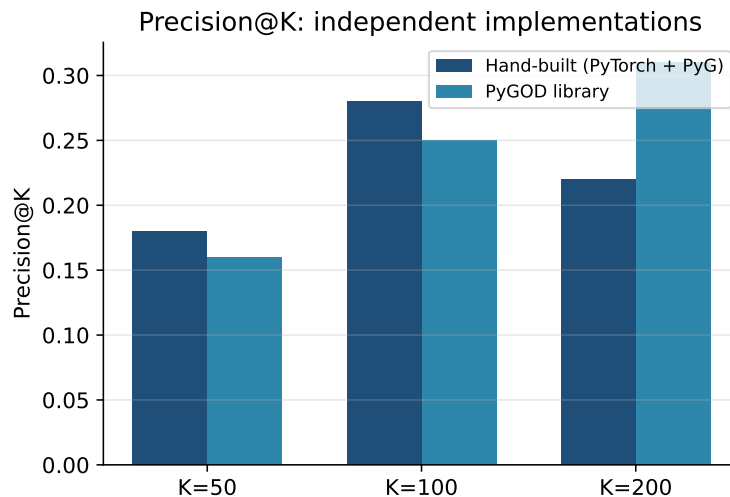


Figure 5: Precision@K for the hand-built implementation versus the PyGOD library implementation, on the same dataset and evaluation protocol.

Notably, the PyGOD library implementation required no manual class-imbalance correction to reach a comparable result, consistent with mature library implementations already accounting for this known failure mode internally by default.

6 Discussion and Next Steps

This reproduction reached the paper’s reported performance range on a real benchmark dataset, confirmed by two independent implementations, and in the process surfaced two genuine implementation-level bugs and one dataset-specific empirical nuance not evident from the paper’s text alone. The debugging process is arguably the more valuable outcome: it demonstrates that DOMINANT’s clean two-decoder narrative — structure and attributes as separate, independently diagnostic signals — does not fully hold on this dataset, where the two signals are entangled by the shared encoder in a way that only becomes visible through careful, labeled diagnostics rather than aggregate metrics alone.

This reproduction is the first part of a small, ongoing project. Planned next steps are: (1) reproduce CoLA [2], a contrastive-learning-based GAD method, as a second and architecturally distinct baseline; and (2) run a generalization experiment — training a detector on one dataset and evaluating it on another — to measure directly how much performance is lost when a specialist model like DOMINANT is applied outside the single graph it was trained on.

References

- [1] K. Ding, J. Li, R. Bhanushali, and H. Liu. *Deep Anomaly Detection on Attributed Networks*. SIAM International Conference on Data Mining (SDM), 2019.
- [2] Y. Liu, Z. Li, S. Pan, C. Gong, C. Zhou, and G. Karypis. *Anomaly Detection on Attributed Networks via Contrastive Self-Supervised Learning*. IEEE Transactions on Neural Networks and Learning Systems, 33(6):2378–2392, 2021.
- [3] K. Liu, Y. Dou, Y. Zhao, X. Ding, X. Hu, R. Zhang, K. Ding, C. Chen, H. Peng, K. Shu, L. Sun, J. Li, G. H. Chen, Z. Jia, and P. S. Yu. *BOND: Benchmarking Unsupervised Outlier Node Detection on Static Attributed Graphs*. Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [4] T. N. Kipf and M. Welling. *Semi-Supervised Classification with Graph Convolutional Networks*. International Conference on Learning Representations (ICLR), 2017.
- [5] T. N. Kipf and M. Welling. *Variational Graph Auto-Encoders*. NeurIPS Bayesian Deep Learning Workshop, 2016.
- [6] K. Liu, Y. Dou, Y. Zhao, X. Ding, X. Hu, R. Zhang, K. Ding, C. Chen, H. Peng, K. Shu, L. Sun, J. Li, G. H. Chen, Z. Jia, and P. S. Yu. *PyGOD: A Python Library for Graph Outlier Detection*. Journal of Machine Learning Research (open-source software track), 2022.